Introducción a la Ciencia de Datos

Maestría en Ciencias de la Computación

Dr. Irvin Hussein López Nava

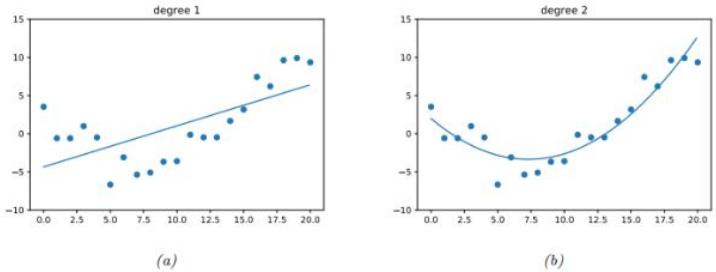


04

Clasificación

Métodos paramétricos

- Construyen una descripción general explícita de la función objetivo f.
  - El modelo resume los datos a través de una colección de parámetros.
  - Número fijo de parámetros.
  - Independiente del número de instancias en los datos de entrenamiento.



Métodos no paramétricos

- No suponen una forma para la función objetivo f.
  - Libertad para elegir cualquier función a partir de los datos de entrenamiento.
  - Requieren muchos más datos para estimar la función objetivo.
- Entradas similares tienen salidas similares.
  - Es una suposición razonable: El mundo es suave y las funciones, ya sean densidades, discriminantes o funciones de regresión, cambian lentamente.

Métodos paramétricos vs no paramétricos

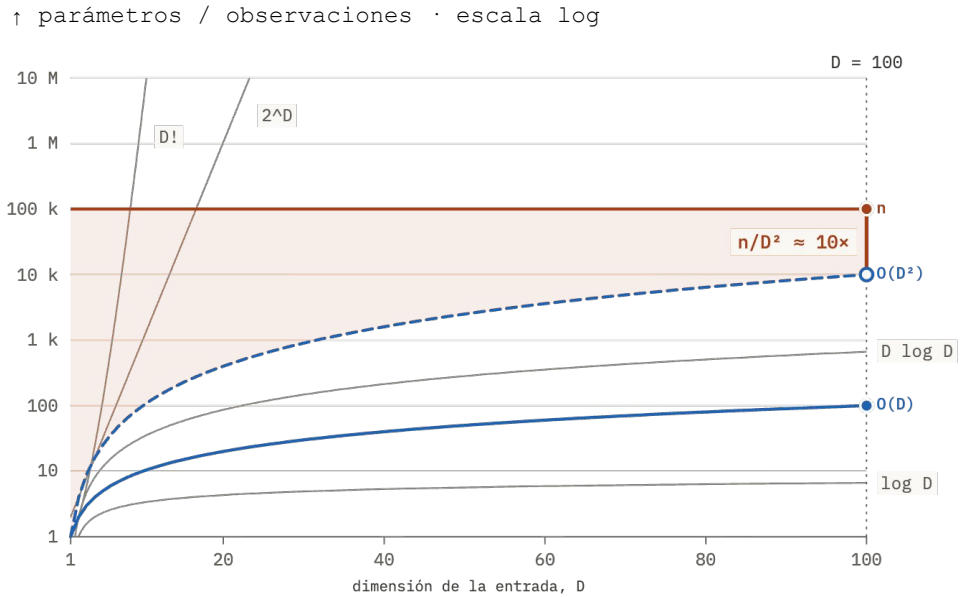
- En un modelo paramétrico, **todas** las instancias de entrenamiento afectan a la estimación global final.
- En un caso no paramétrico, no existe un modelo global único;
  - los modelos locales se estiman a medida que se necesitan, y sólo se ven afectados por las instancias de entrenamiento cercanas.

Métodos paramétricos vs no paramétricos

- En ML, los métodos **no paramétricos** también se denominan algoritmos de aprendizaje basados en instancias o memoria, ya que almacenan las instancias de entrenamiento en una tabla de consulta e “interpolan” a partir de ella.
  - Dado que todas las instancias de entrenamiento deben ser almacenadas, se requieren memoria $O(n)$.
  - Dada una entrada, encontrar otras similares requiere el cálculo de $O(n)$.

Métodos paramétricos vs no paramétricos

- En un **enfoque paramétrico** el modelo puede ser bastante simple, con pocos parámetros: del orden de O(D) u O(D²), donde D es la dimensión de la entrada.
- En general, el número de observaciones n es mucho mayor: n ≫ D, D².



Métodos paramétricos vs no paramétricos

- Los métodos **basados en instancias** también se llaman perezosos (*lazy*): a diferencia de los ansiosos (*eager*), no calculan un modelo al recibir el conjunto de entrenamiento, sino que posponen el cálculo hasta recibir una consulta.
  - No todo método no paramétrico es perezoso: el árbol de la sesión 13 crece con los datos y es ansioso.

Tell me about your friends, and ...











... I will tell you who you are.

4.4 Instance-based learning


¿Qué es el
Aprendizaje basado
en Instancias?

Definiciones

ChatGPT

Es un enfoque en aprendizaje automático que se centra en la utilización de ejemplos específicos o "instancias" para realizar tareas de clasificación. Se basa en la retención y recuperación de casos o ejemplos individuales para resolver problemas.

https://chat.openai.com/

Wikipedia

Es una familia de algoritmos de aprendizaje que, en lugar de realizar una generalización explícita, compara nuevas instancias de problemas con instancias vistas en el entrenamiento, que han sido almacenadas en la memoria.

https://en.wikipedia.org/wiki/Instance-based_learning

Gemini

Es un enfoque de aprendizaje automático que almacena ejemplos de datos de entrenamiento y los utiliza para realizar predicciones.
Es diferente del aprendizaje basado en modelos, que construye un modelo matemático de los datos de entrenamiento.

https://gemini.google.com/

¿Cuándo optar por este enfoque?

- Menos de 20 atributos por instancia.
- Muchos datos de entrenamiento.

Ventajas:

- El entrenamiento es muy rápido.
- Aprende funciones objetivo complejas.
- No se pierde información.

Desventajas:

- Lento en el momento de la consulta.
- Fácilmente engañado por atributos irrelevantes.
- Sensible a la escala de los atributos.

Idea principal

Vecino más próximo (*nearest neighbor*):

- Se localiza el ejemplo de entrenamiento más cercano a la consulta x_q, x_n, y se copia su valor.

$$\hat{f}(x_q) \leftarrow f(x_n)$$

k vecinos más próximos (*k-nearest neighbor*):

- Si *f* es discreta, votan los *k* más cercanos; δ (*a*, *b*) vale 1 si *a* = *b* y 0 si no.

$$\hat{f}(x_q) \leftarrow \arg\max_{v \in V} \sum_{i=1}^{k} \delta(v, f(x_i))$$

- Si *f* es real, se promedian.

$$\hat{f}(x_q) \leftarrow \frac{1}{k} \sum_{i=1}^{k} f(x_i)$$

Suposiciones iniciales

Todas las instancias corresponden a puntos de un espacio de D dimensiones, $\mathbb{R}^D$.

- Una instancia x se describe con el vector de atributos $(a_1(x), a_2(x), \ldots, a_D(x))$; $a_r(x)$ es el valor del r–ésimo atributo.
- Los vecinos más cercanos se pueden definir con la distancia euclidiana:

$$d(x_i, x_j) = \sqrt{\sum_{r=1}^{D} (a_r(x_i) - a_r(x_j))^2}$$

***Nota**: La suma mezcla diferencias con unidades distintas, el atributo de mayor escala decide quiénes son los vecinos.*

Algoritmo base kNN

Algoritmo k vecinos más cercanos (kNN)

1: **Entrada:** ejemplos de entrenamiento $T = \{(x_1, f(x_1)), \ldots, (x_n, f(x_n))\}$, consulta x_q, vecinos k
2: **Salida:** la clase estimada $\hat{f}(x_q)$
3: **para cada** ejemplo x_i en T **hacer**
4: calcular la distancia $d(x_q, x_i)$
5: ordenar los ejemplos de menor a mayor distancia
6: tomar los k primeros
7: contar cuántos de esos k hay de cada clase
8: **devolver** la clase con más votos

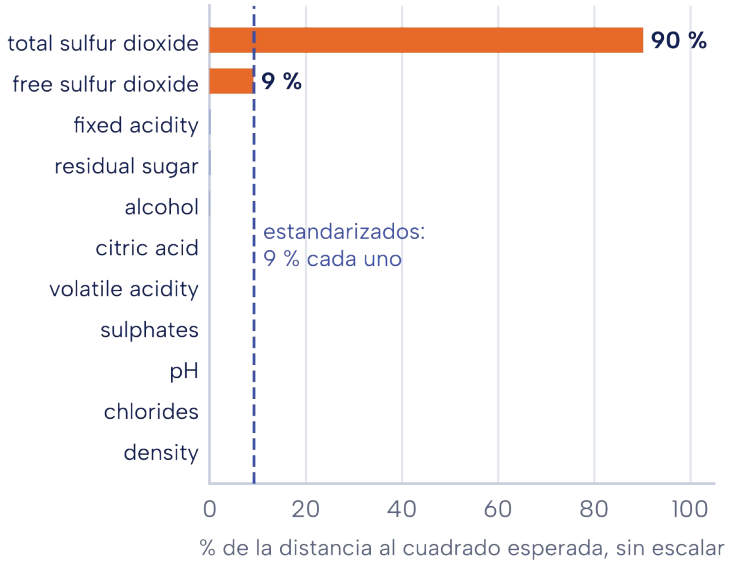
Entrenar es guardar T: cada consulta cuesta O(nD). Con dos clases y k impar no hay empates en el voto; scikit-learn los da a la primera clase en orden alfabético.

¡La elección de la medida de distancia afecta!

Con $\Delta_r = a_r(x_i) - a_r(x_j)$:

Distancia	Fórmula	metric en scikit-learn
Euclidiana	$\sqrt{\sum_r \Delta_r^2}$	`"euclidean" (p = 2)`
Manhattan	$\sum_r \lvert\Delta_r\rvert$	`"manhattan" (p = 1)`
Minkowski	$\left(\sum_r \lvert\Delta_r\rvert^p\right)^{1/p}$	`"minkowski", p (por omisión, p = 2)`
Chebyshev	$\max_r \lvert\Delta_r\rvert$	`"chebyshev" (p → ∞)`
Coseno	$1 - \frac{x_i \cdot x_j}{\lVert x_i\rVert \, \lVert x_j\rVert}$	`"cosine": 1 - la similitud`
Hamming	$\sum_r [\, a_r(x_i) \neq a_r(x_j)\,]$	`"hamming": la fracción, entre D`

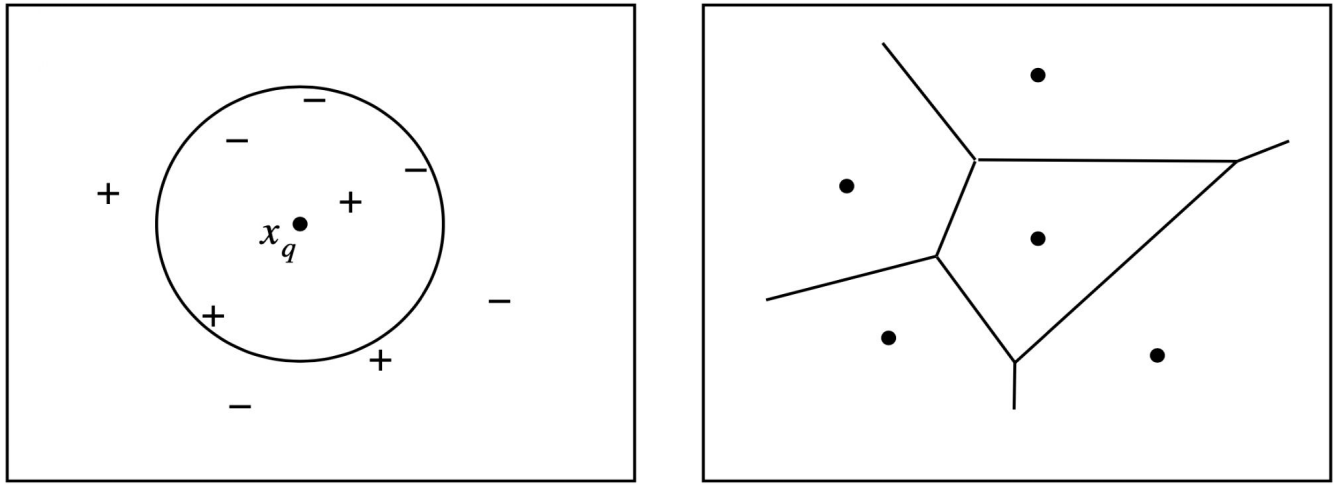
La escala decide quiénes son los vecinos



- El SO_2 total va de 6 a 289; la densidad, de 0.990 a 1.004.
- Sin escalar, el SO_2 total aporta 90 % de la distancia, y con el SO_2 libre, 99 %.
- Los 10 vecinos de un vino de prueba coinciden en 64 % con los que dan solo las dos columnas de SO_2, y en 11 % con los de los datos estandarizados.
- F1 macro en validación, $k = 15$: 0.45 sin escalar y 0.52 estandarizado.

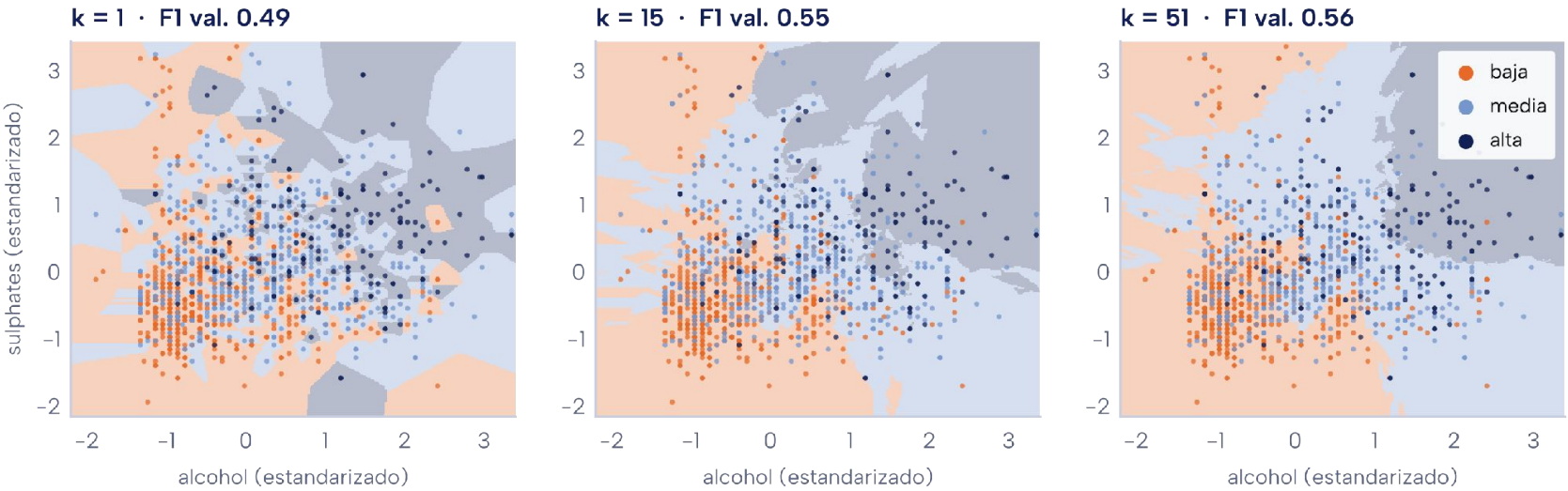
El escalador va dentro del `Pipeline`: cada pliegue aprende la escala solo de su entrenamiento.

Diagrama de Voronoi



Con 1 vecino, x_q es +; con 5, −. A la derecha, la región de cada ejemplo con 1 vecino.

Límites de decisión inducidos por kNN



El vino con dos atributos estandarizados. Con $k = 1$, cada vino de entrenamiento crea su propia isla; al crecer k, las fronteras se suavizan.

Nuestro ejemplo con vecinos

La instancia de prueba de las sesiones 13 y 14: {*sunny, cool, high, strong*}. Distancia de Hamming: cuántos atributos no coinciden.

***d* hamming**	**Días No**	**Días Sí**
1	2	—
2	1, 6, 8, 14	7, 9, 11, 12
3	—	3, 4, 5
4	—	10, 13

- $k = 1$: el día 2 → **No**.
- $k = 3$: el día 2 y dos de los ocho a distancia 2: **empate**. En 200 órdenes al azar, Sí en 53.
- $k = 9$: todos a distancia ≤ 2 → **No**, 5 a 4.
- Los 14, con $w = 1/d^2$: **No** (0.578); con $w = 1/d$: **Sí** (0.538).

El árbol (13) y Naive Bayes (14) también dicen No. Pero dejando uno fuera, el 1 vecino acierta 6 de 14 días, y responder siempre Sí acierta 9.

Caso de estudio: 14 días más

Entrenamiento: los días de las sesiones 13 y 14

Día	Outlook	Temp.	Hum.	Wind	Play
1	sunny	85	85	weak	No
2	sunny	80	90	strong	No
3	overcast	83	86	weak	Sí
4	rain	70	96	weak	Sí
5	rain	68	80	weak	Sí
6	rain	65	70	strong	No
7	overcast	64	65	strong	Sí
8	sunny	72	95	weak	No
9	sunny	69	70	weak	Sí
10	rain	75	80	weak	Sí
11	sunny	75	70	strong	Sí
12	overcast	72	90	strong	Sí
13	overcast	81	75	weak	Sí
14	rain	71	91	strong	No

Prueba: 14 días nuevos (Weather.prueba.csv)

Día	Outlook	Temp.	Hum.	Wind	Play
15	sunny	81	88	strong	No
16	overcast	74	92	weak	Sí
17	rain	76	85	weak	Sí
18	sunny	78	75	strong	No
19	sunny	82	92	weak	No
20	rain	67	90	strong	No
21	overcast	85	85	strong	Sí
22	rain	73	88	weak	Sí
23	sunny	88	65	strong	Sí
24	overcast	77	70	weak	Sí
25	sunny	79	60	weak	Sí
26	overcast	80	95	strong	Sí
27	rain	66	70	weak	No
28	overcast	84	78	weak	Sí

Codificar y escalar

- **outlook** → one-hot: tres columnas 0/1. Con códigos 0, 1 y 2, sunny quedaría a 2 de rain y a 1 de overcast: un orden falso. Con one-hot, todas las parejas quedan a $\sqrt{2}$.
- **wind** → 0/1.
- **temp.** y **hum.** → $z = (x - \mu)/\sigma$, con μ y σ **del entrenamiento**: 73.57 y 6.33 para la temperatura, 81.64 y 9.91 para la humedad. La prueba se escala con esos mismos valores.

Día 1: sunny, 85, 85, weak
Día 15: sunny, 81, 88, strong

Día	sunny	overc.	rain	z temp	z hum	wind
1	1	0	0	1.80	0.34	0
15	1	0	0	1.17	0.64	1

Sin escalar, 10 grados de diferencia suman 100 a d^2; un cambio de viento suma 1.

El día 15 y sus vecinos

Consulta: día 15 (sunny, 81, 88, strong). Su clase real es **No**.

#	Día	d	Play
1	2	0.26	No
2	1	1.22	No
3	3	1.77	Sí
4	8	1.88	No
5	12	2.02	Sí
6	11	2.05	Sí
7	10	2.13	Sí
8	14	2.14	No
…			

k	votos No	votos Sí	predicción
1	1	0	**No**
3	2	1	**No**
5	3	2	**No**
7	3	4	**Sí**

*Con k = 5 gana **No**, 3 a 2.*
*Con k = 7 entran dos **Sí** más y la predicción cambia: k decide.*

Los 14 días de prueba

Aciertos de 14	k = 1	k = 3	k = 5	k = 7
con z-scores	13	11	10	8
sin escalar	11	7	7	10

- Con $k = 7$, 13 de los 14 días salen Sí: la clase mayoritaria se impone.
- Día 21 (overcast, 85, 85, strong; Sí).
  - Sin escalar, su vecino más cercano es el día 1 (sunny, 85, 85, weak; No), a 1.73: coinciden en temperatura y humedad, y eso pesa más que outlook y wind.
  - Con z-scores es el día 3 (overcast, 83, 86, weak; Sí), a 1.05.

Elegir *k* sin mirar la prueba

- Con la prueba, gana $k = 1$: 13/14 (93 %). Pero ese número es optimista: la prueba ya decidió.
- Lo correcto: elegir *k* con validación cruzada dentro del entrenamiento, reentrenar con los 14 días y evaluar una sola vez.
- Dejando uno fuera, cada día se clasifica con los otros 13. Empatan $k = 5$ y $k = 7$; con el menor, la prueba da **10/14 (71 %)**.

k	Aciertos dejando uno fuera
1	8/14
3	7/14
5	**9/14**
7	**9/14**

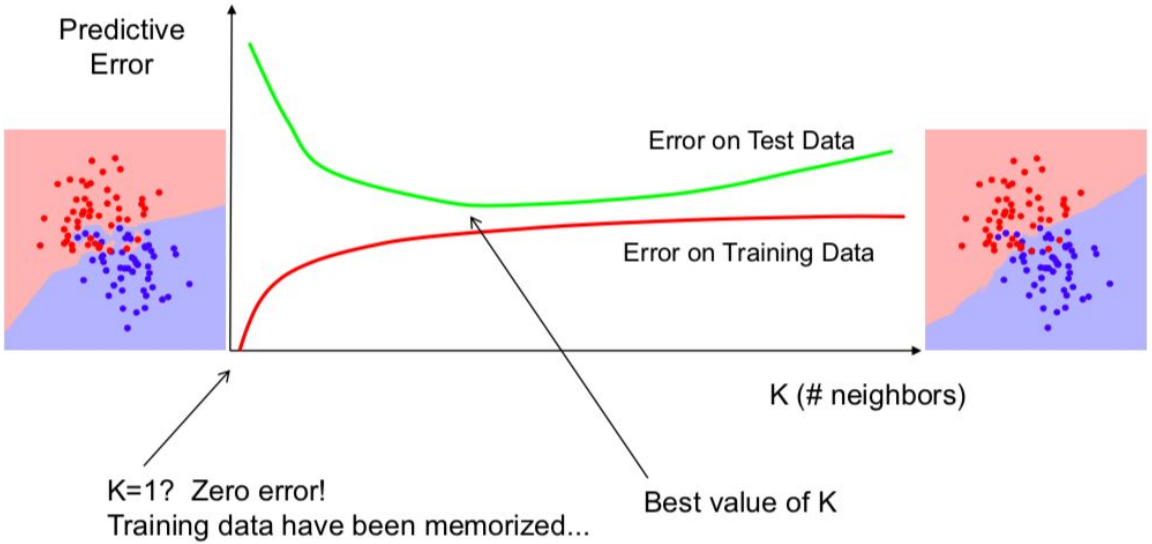
Con 14 días, un acierto vale 7 puntos: toda estimación es incierta. μ y σ se recalculan en cada partición, dentro del `Pipeline`.

La unidad en los mismos 14 días

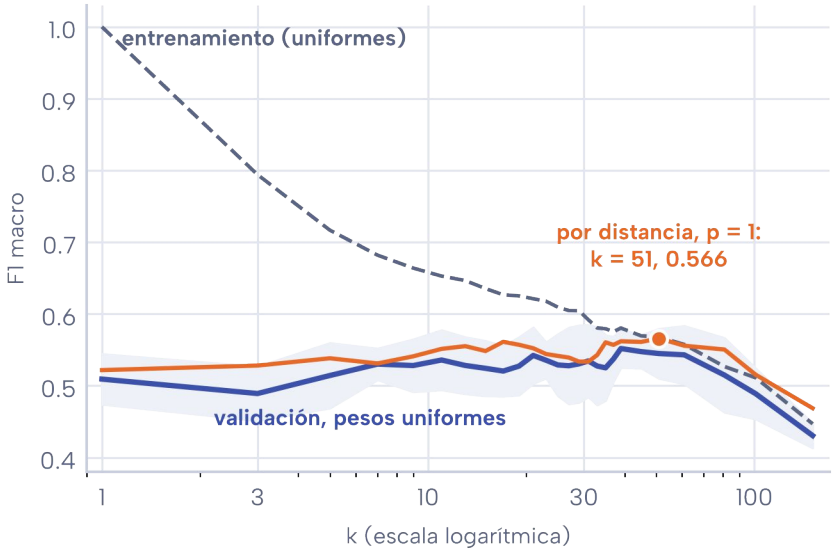
Modelo	Aciertos	Exactitud
Árbol J48 (sesión 13)	12/14	86 %
Naive Bayes, una normal por clase (sesión 14)	12/14	86 %
kNN, k = 5, elegido sin mirar la prueba	10/14	71 %
kNN, k = 1, elegido con la prueba	13/14	93 % (optimista)

- Un día de prueba vale 7 puntos: ninguna diferencia es concluyente.
- La comparación seria se hace con muchos más datos: el vino, con 1599 vinos, validación cruzada agrupada y la prueba una sola vez.

Elegir el valor de k



Elegir *k* con validación: el vino



- En entrenamiento, $k = 1$ acierta todo: memoriza.
- La validación sube hasta $k \approx 10$ y se mantiene entre 0.52 y 0.57 hasta $k \approx 60$; entre pliegues varía de 0.02 a 0.05.
- Gana $k = 51$ con Manhattan y pesos por distancia: 0.566. En prueba, una sola vez: **0.584**.

Naive Bayes 0.596, logística 0.573 y árbol 0.551 (notebook 14): las diferencias no superan el ruido. El log-loss del kNN, 0.77, se parece al de la logística (0.75).

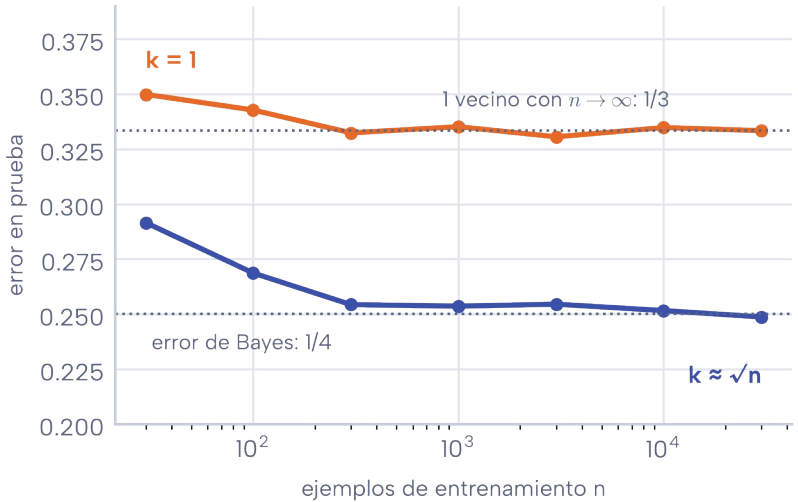
Comportamiento en el límite

Sea $p(x)$ la probabilidad de que x tenga la etiqueta 1.

- **1 vecino:** cuando $n \to \infty$, se aproxima al algoritmo de Gibbs, que predice 1 con probabilidad $p(x)$. Su error es a lo más el doble del de Bayes, R^*:

$$R^* \leq R_{1\mathrm{NN}} \leq 2R^*(1 - R^*)$$

- ***k* vecinos:** cuando $n \to \infty$ y k crece más despacio que n, se aproxima al óptimo de Bayes: predice 1 si $p(x) > 0.5$.



x uniforme en [0, 1] y p(x) = x: el error de Bayes es 1/4; el del 1 vecino se queda en 1/3 aunque crezca n.

Distance-Weighted kNN

Podría quererse ponderar más a los vecinos cercanos. Si *f* es discreta y si es real:

$$\hat{f}(x_q) \leftarrow \arg\max_{v \in V} \sum_{i=1}^{k} w_i \, \delta(v, f(x_i))$$

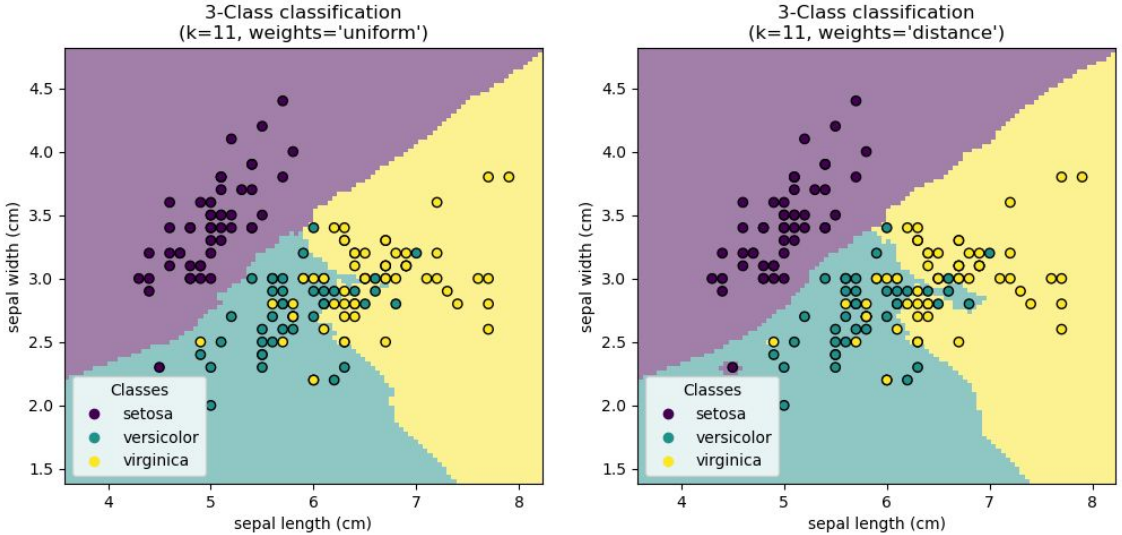
$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^{k} w_i \, f(x_i)}{\sum_{i=1}^{k} w_i}$$

donde $$w_i \equiv \frac{1}{d(x_q, x_i)^2}$$

En scikit-learn, `weights="distance"` usa $w_i = 1/d$, no $1/d^2$.

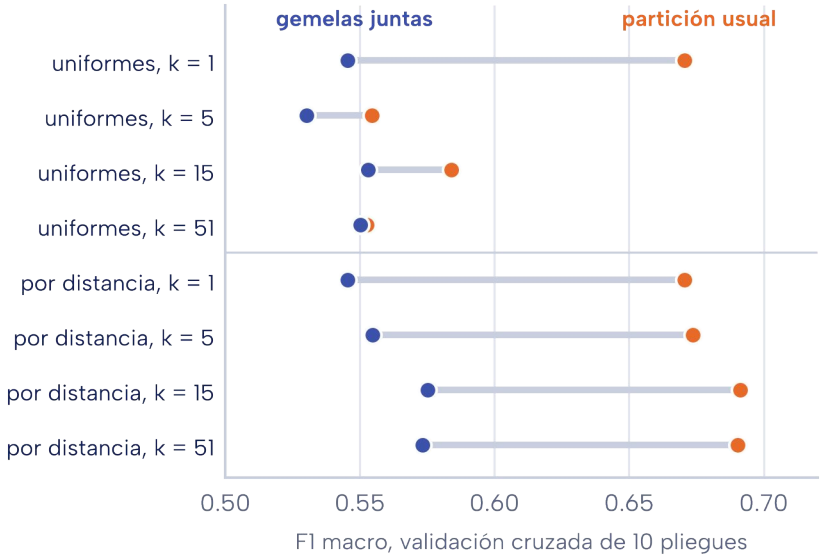
- Si $d(x_q, x_i) = 0$, se copia $f(x_i)$: un duplicado decide solo.
- Ahora tiene sentido usar todos los ejemplos de entrenamiento en lugar de solo *k* (método de Shepard).

Límites de decisión inducidos por KNN



k = 11 con voto uniforme (izquierda) y ponderado por distancia (derecha).
En scikit-learn, `weights='distance'` pondera con 1/*d*.

Los duplicados y los pesos por distancia



- 240 filas duplicadas: con la partición usual, 26 % de los vinos de prueba tienen su gemela en entrenamiento.
- Con pesos uniformes, el efecto se diluye al crecer *k*: con k = 51, la diferencia es de 0.003.
- Con pesos por distancia, la gemela está a distancia 0 y decide sola con cualquier *k*. Con *k* = 15: 0.69 con la partición usual, más que cualquier modelo de la unidad, y 0.58 con las gemelas juntas.

Por la misma razón, con pesos por distancia el F1 de entrenamiento vale 1 con cualquier k.

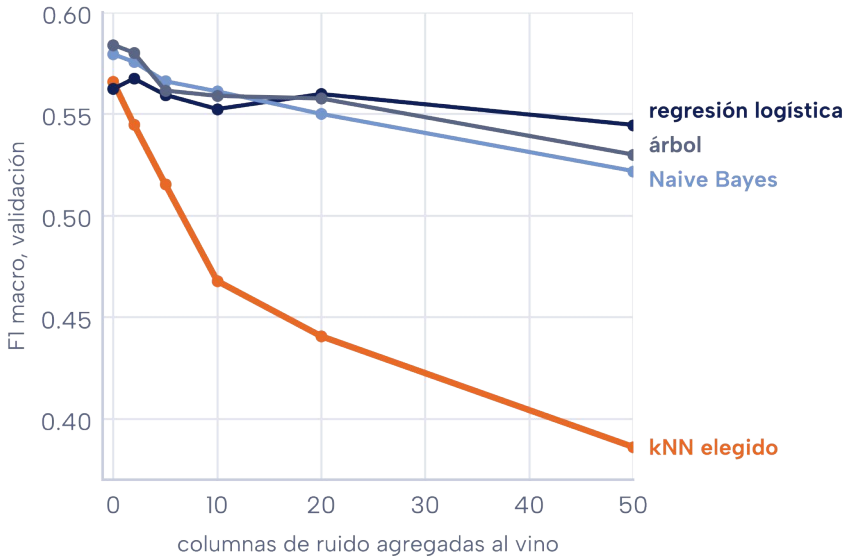
kNN en scikit-learn

Parámetro	Por omisión	Qué controla
`n_neighbors`	`5`	k; con dos clases, mejor impar
`weights`	`"uniform"`	"distance" usa 1/d
`metric, p`	`"minkowski", 2`	la euclidiana; p = 1 da Manhattan
`algorithm`	`"auto"`	k-d tree, ball tree o fuerza bruta

- `make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))`: el escalador primero (lámina 21). En regresión, `KNeighborsRegressor` (lámina 46).
- Un empate en el voto lo gana la primera clase en orden alfabético. Con $k = 2$, empata 36 % de la prueba del vino, y ninguno se resuelve a favor de «media».
- `predict_proba` es la fracción de votos: con $k = 1$ solo hay 0 y 1, y el log-loss en prueba es de 14.6.

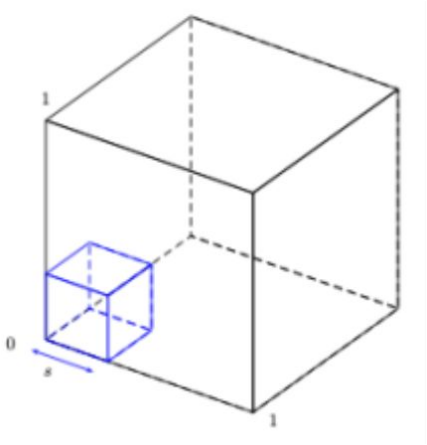
Maldición de la dimensionalidad

- El principal problema estadístico de kNN: con muchas dimensiones no funciona bien.
- Si de 20 atributos solo 2 son relevantes, la distancia la deciden los otros 18.
- En el vino, con 20 columnas de ruido, el kNN elegido baja de **0.57** a **0.44** en validación. La logística, Naive Bayes y el árbol pierden 0.03 o menos.
- *Curse of dimensionality*: el vecino más cercano se equivoca fácilmente cuando *X* tiene muchas dimensiones.



Longitud de borde esperada

- Suponga que se aplica un clasificador *k*NN a datos en los que las entradas se distribuyen uniformemente en un cubo unitario D–dimensional.
- Ahora, suponga que se estima la densidad de etiquetas de clase alrededor de un punto de prueba $\boldsymbol{x}$ haciendo crecer un hipercubo alrededor de $\boldsymbol{x}$ hasta que contenga una fracción deseada p de los puntos de datos.

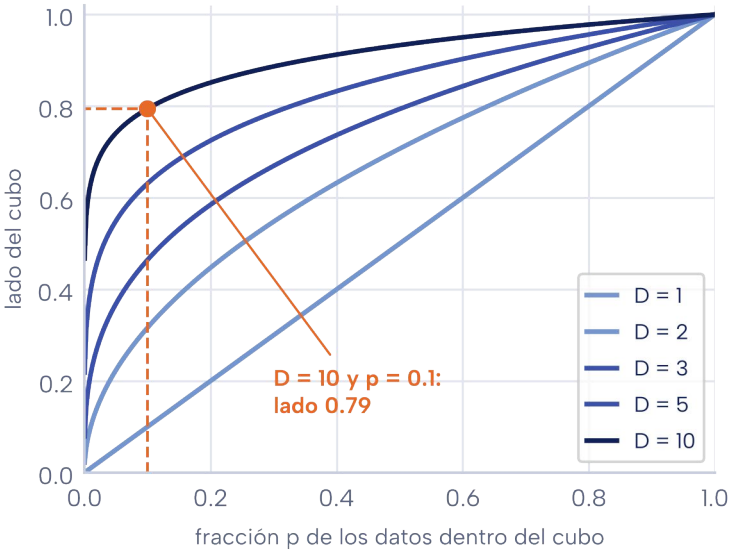


Longitud de borde esperada

- La longitud de borde esperada del cubo es:

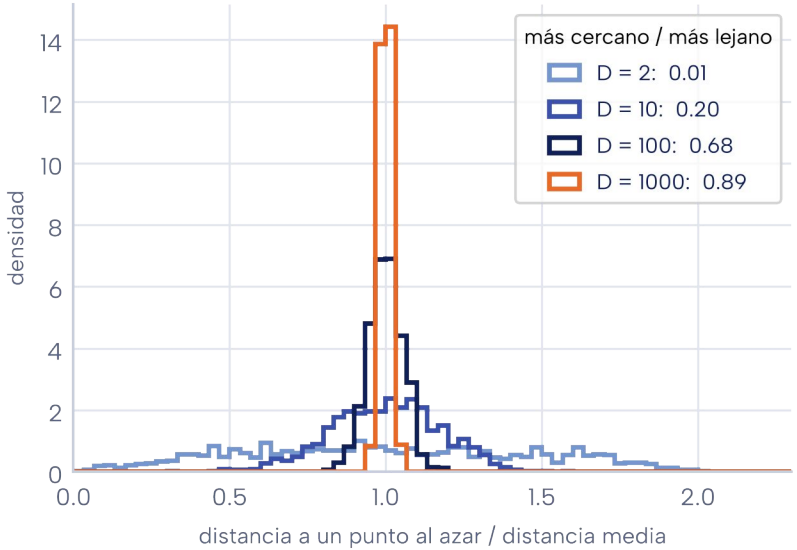
$$e_D(p) = p^{1/D}$$

- Con D = 10 y 10 % de los datos, $e_{10}(0.1)$ = 0.79: el cubo debe abarcar 79 % de cada eje alrededor de x. Con 1 %, $e_{10}(0.01)$ = 0.63.
- En el vino, 51 vecinos de 1279 en 11 dimensiones necesitarían 75 % de cada eje, si los datos fueran uniformes.



Las distancias se concentran

- Se generan 1000 puntos al azar en el cubo unitario de *D* dimensiones y se mide su distancia a otro punto al azar.
- El más cercano está a 1 % de la distancia del más lejano con *D* = 2; a 20 % con 10, a 68 % con 100 y a 97 % con 10 000.
- Con muchas dimensiones, todos los puntos quedan casi a la misma distancia: el vecino más cercano apenas está más cerca que los demás.



Contra la maldición: pesar los atributos

- Estirar el eje r con un peso z_r:

$$d_z(x_i, x_j) = \sqrt{\sum_{r=1}^{D} z_r^2 \left(a_r(x_i) - a_r(x_j)\right)^2}$$

- $z_1, \ldots, z_D$ se eligen con validación cruzada; $z_r = 0$ elimina el atributo.
- Estandarizar es $z_r = 1/s_r$: un peso fijo, que no mira la clase.
- La selección de atributos de la sesión 9 usa $z_r \in \{0, 1\}$. Con las 20 columnas de ruido, la selección hacia adelante las descarta todas, pero deja fuera 7 de los 11 atributos reales: la prueba sube de 0.50 a 0.55, no a 0.58.
- Otro enfoque: reducir la dimensionalidad (PCA, sesión 8).

Reducción de memoria y tiempo

- **Memoria:** kNN guarda todo T. Basta guardar los puntos cercanos a las fronteras de decisión: los del interior no cambian ninguna decisión (e.g., *condensed nearest neighbor*; Hart, 1968).
- **Tiempo:** comparar la consulta con los n ejemplos cuesta $O(nD)$. El reto es encontrar los k vecinos en menos de $O(n)$.
- Partición del espacio: k-d tree y ball tree, las opciones de `algorithm` en scikit-learn. Con pocas dimensiones son mucho más rápidos: con 20 000 puntos y $D = 2$, un orden de magnitud.
- Con muchas dimensiones pierden su ventaja: con $D = 10$ ya son más lentos que comparar contra todos. Por eso `algorithm="auto"` usa fuerza bruta si $D > 15$.
- Con millones de puntos se buscan vecinos aproximados, por ejemplo con hashing sensible a la localidad (LSH).

Regresión ponderada local

- kNN forma una aproximación local a *f* para cada consulta x_q.
- ¿Por qué no ajustar una aproximación explícita $\hat{f}(x)$ en la región que rodea a x_q?
  - Una función lineal con los *k* vecinos más cercanos:

$$\hat{f}(x) = w_0 + w_1\, a_1(x) + \cdots + w_D\, a_D(x)$$

  - Un ajuste cuadrático, …
- Produce una aproximación «por partes» de *f*: una recta distinta para cada consulta.

Regresión ponderada local

- Varias opciones de error a minimizar:
  - error cuadrático sobre los *k* vecinos más cercanos:

$$E_1(x_q) \equiv \frac{1}{2} \sum_{x \in k \text{ vecinos de } x_q} \left(f(x) - \hat{f}(x)\right)^2$$

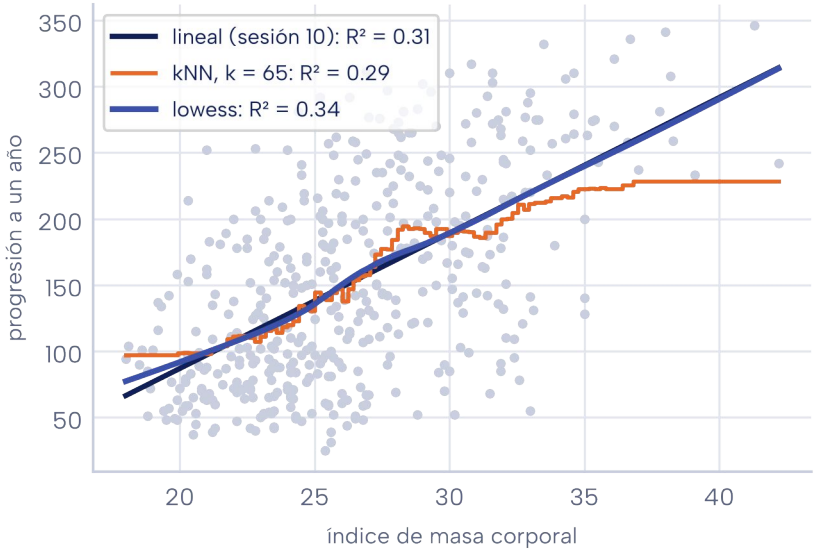
  - ponderado por la distancia, sobre todo el entrenamiento *T*:

$$E_2(x_q) \equiv \frac{1}{2} \sum_{x \in T} \left(f(x) - \hat{f}(x)\right)^2 K(d(x_q, x))$$

  - ponderado, pero solo sobre los *k* vecinos: su costo no depende de *n*:

$$E_3(x_q) \equiv \frac{1}{2} \sum_{x \in k \text{ vecinos de } x_q} \left(f(x) - \hat{f}(x)\right)^2 K(d(x_q, x))$$

Regresión local con la diabetes



- Los 442 pacientes de la sesión 10, con un solo atributo: el IMC.
- R^2 con validación cruzada: lineal **0.31**, kNN **0.29** y *lowess*, que ajusta una recta ponderada en cada punto, **0.34**.
- Con los diez atributos, la lineal da **0.46** y el kNN 0.43 estandarizado y 0.29 sin escalar.
- Si la relación es casi lineal, el método perezoso no gana nada.

Lazy and eager learning

Lazy: espera a la consulta antes de generalizar.

- Vecino más cercano, razonamiento basado en casos.

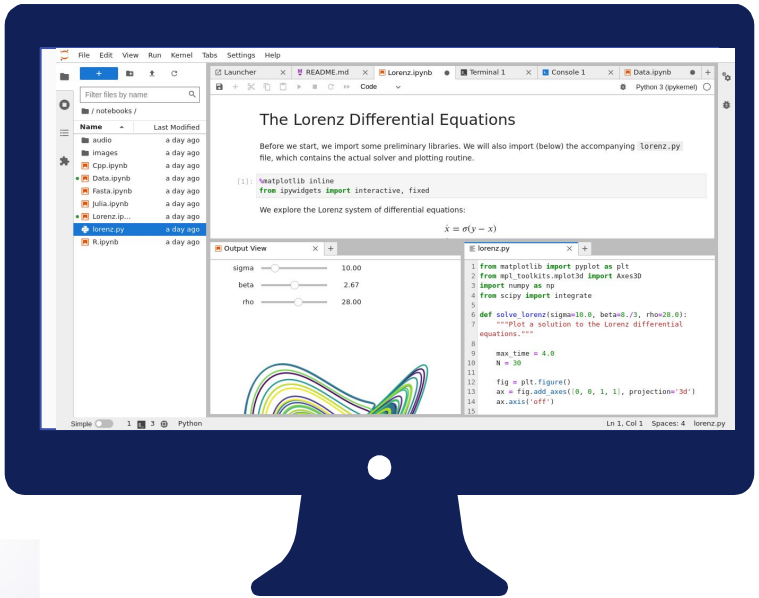
Eager: generaliza antes de ver la consulta.

- Redes de funciones de base radial, ID3, retropropagación, Naive Bayes, ...

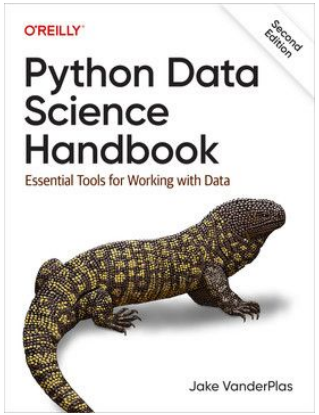
¿Importa?

- El aprendiz ansioso debe crear una aproximación global.
- El aprendiz perezoso puede crear muchas aproximaciones locales.
- Si usan la misma H, los perezosos pueden representar funciones más complejas.

(Go to live notebook)



Extra Libro



GitHub

?
02.08-Sorting.ipynb

Gracias!

¿Alguna pregunta?

hussein@cicese.edu.mx



SCAN ME